

CODING CHALLENGE

CO4 - AT1

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

INPUT

```
# Define a small test dataset
X_test = np.array([
    [1.2, 1.0],
    [6.0, 9.0],
    [0.5, 0.5],
    [7.5, 7.5]
])
y_true = np.array([0, 1, 0, 1]) # True labels for the test set

# Get K from the user
while True:
    try:
        k = int(input("Enter the value of K (an odd integer is recommended, e.g., 3 or 5): "))
        if k > 0:
            break
        else:
            print("K must be a positive integer.")
    except ValueError:
        print("Invalid input. Please enter an integer.")

print(f"\nUsing K = {k}")

predictions = []
correct_predictions = 0

for i, test_instance in enumerate(X_test):
    predicted_class = predict_classification(X_train, y_train, test_instance, k)
    predictions.append(predicted_class)
    is_correct = (predicted_class == y_true[i])
    if is_correct:
        correct_predictions += 1
    print(f"Test instance: {test_instance}, True label: {int(y_true[i])}, Predicted label: {predicted_class}")

accuracy = (correct_predictions / len(X_test)) * 100
print(f"\nPredicted Labels: {predictions}")
print(f"Accuracy: {accuracy:.2f}%")
```

OUTPUT:

```
Enter the value of K (an odd integer is recommended, e.g., 3 or 5): 3

Using K = 3
Test instance: [1.2 1. ], True label: 0, Predicted label: 0, Correct: True
Test instance: [6. 9.], True label: 1, Predicted label: 1, Correct: True
Test instance: [0.5 0.5], True label: 0, Predicted label: 0, Correct: True
Test instance: [7.5 7.5], True label: 1, Predicted label: 1, Correct: True

Predicted Labels: [0, 1, 0, 1]
Accuracy: 100.00%
```

2. Write a Python program to implement **Logistic Regression** for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

INPUT

```
import numpy as np

# Binary classification dataset
X = np.array([[1,2],[2,3],[3,4],[4,5],[5,6],
              [6,7],[7,8],[8,9],[9,10],[10,11]], dtype=float)
y = np.array([0,0,0,0,0,1,1,1,1,1])

# Add bias column
X = np.c_[np.ones(X.shape[0]), X]

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def train(X, y, lr, epochs=1000):
    w = np.zeros(X.shape[1])
    for _ in range(epochs):
        p = sigmoid(X @ w)
        gradient = X.T @ (p - y) / len(y)
        w -= lr * gradient
    return w

# Train with different learning rates
for lr in [0.001, 0.01, 0.1, 1.0]:
    w = train(X, y, lr)
    prob = sigmoid(X @ w)
    pred = (prob >= 0.5).astype(int)
    accuracy = np.mean(pred == y) * 100

    print("\nLearning Rate:", lr)
    print("Sigmoid Outputs:", np.round(prob, 3))
    print("Predictions:      ", pred)
    print("Actual:           ", y)
    print("Accuracy: %.2f%%" % accuracy)

# Test data
test = np.array([[1, 2.5], [1, 5.5], [1, 8.5]])
```

OUTPUT

```
Learning Rate: 0.001
Sigmoid Outputs: [0.499 0.538 0.576 0.613 0.649 0.683 0.715 0.746 0.774 0.8 ]
Predictions:      [0 1 1 1 1 1 1 1 1 1]
Actual:           [0 0 0 0 0 1 1 1 1 1]
Accuracy: 60.00%

Learning Rate: 0.01
Sigmoid Outputs: [0.215 0.29  0.378 0.476 0.575 0.668 0.75  0.817 0.87  0.909]
Predictions:      [0 0 0 0 1 1 1 1 1 1]
Actual:           [0 0 0 0 0 1 1 1 1 1]
Accuracy: 90.00%

Learning Rate: 0.1
Sigmoid Outputs: [0.006 0.019 0.06  0.175 0.411 0.696 0.883 0.961 0.988 0.996]
Predictions:      [0 0 0 0 0 1 1 1 1 1]
Actual:           [0 0 0 0 0 1 1 1 1 1]
Accuracy: 100.00%

Learning Rate: 1.0
Sigmoid Outputs: [0.    0.    0.    0.003 0.127 0.893 0.998 1.    1.    1.    ]
Predictions:      [0 0 0 0 0 1 1 1 1 1]
Actual:           [0 0 0 0 0 1 1 1 1 1]
Accuracy: 100.00%
```